

TrekEasy

DevOps & AWS ECS on Fargate — Complete Documentation

[Project & DevOps Overview](#) • [Manual Runbook](#)

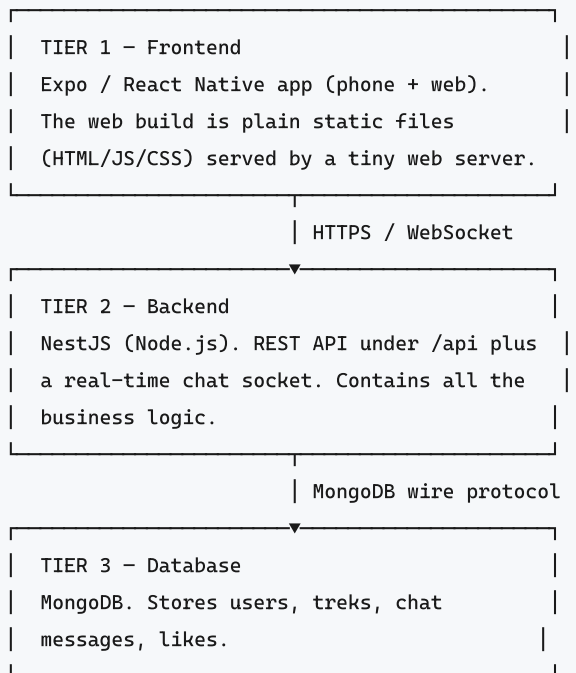
Part 1 — Project & DevOps Overview

TrekEasy — Project & DevOps Overview

A plain-English tour of how TrekEasy is built, containerized, tested, and deployed. No prior DevOps knowledge assumed. If you just want the commands, skip to [RUNBOOK.md](#).

1. What the application is

TrekEasy is a trek-discovery app for the Himalayas: personalized recommendations, trending treks, and a group chat per expedition. It is a **3-tier application**:



"3-tier" simply means the **presentation** (frontend), the **logic** (backend), and the **data** (database) are three separate programs that can be scaled, deployed, and reasoned about independently.

Repository layout

Folder	Tier	What it holds
<code>frontend/</code>	1	Expo app — <code>app/</code> (screens), <code>components/</code> , <code>lib/</code> (API client), <code>context/</code>
<code>backend/</code>	2	NestJS API — <code>src/modules/{auth,users,destinations,chat,recommendations,itinerary,health}</code>
<code>backend-database/</code>	2→3	Mongoose schemas + DB/JWT config. Not a running server — it compiles <i>into</i> the backend image. MongoDB is the actual tier-3 service.
<code>deploy/</code>	—	<code>nginx/</code> reverse-proxy config, <code>apache/</code> alternative
<code>k8s/</code>	—	Kubernetes manifests (Minikube)
<code>aws/</code>	—	ECS on Fargate — scripts that ship new versions onto the already-running cluster/service
<code>.github/workflows/</code>	—	GitHub Actions CI and CD
<code>Jenkinsfile</code>	—	Equivalent pipeline for a Jenkins server
<code>docker-compose.yml</code>	—	Run all four containers locally with one command

How the frontend finds the backend

The frontend is compiled with `EXPO_PUBLIC_API_URL=/api` **baked in** — a *relative* path. When the browser loads the app from `http://some-host/`, it resolves API calls against **that same host** (`http://some-host/api/...`). That is why every deployment (nginx, Kubernetes Ingress, AWS ALB) puts the frontend and backend behind **one hostname** and routes by path. Same origin = no CORS headaches, and the same image works in every environment.

Standardized names

One naming scheme for frontend/backend everywhere — Compose, Kubernetes, and AWS:

Tier	Image	Container / task	Network name
frontend	<code>trekeasy-frontend</code>	<code>trekeasy-frontend</code>	<code>frontend</code>
backend	<code>trekeasy-backend</code>	<code>trekeasy-backend</code>	<code>backend</code>
database (Compose / K8s only)	<code>trekeasy-database</code>	<code>trekeasy-database</code>	<code>mongo</code>

The database's *network name* stays `mongo` on purpose in Compose and Kubernetes — it is hard-coded into the connection string `mongodb://mongo:27017/trekeasy` in both places (Compose, `k8s/db-secret.yaml`). Renaming a load-bearing hostname buys nothing; the **image** and **container** names carry the brand. AWS has no `trekeasy-database` container at all — the database tier there is the managed **Amazon DocumentDB** service (see §5b).

2. Docker & Dockerfiles

What a container is

A **container** is your app plus exactly the runtime and libraries it needs, packaged into a single read-only bundle called an **image**. The same image runs identically on your laptop, in CI, and on AWS — "works on my machine" stops being a problem. A **Dockerfile** is the recipe for building an image.

The four images

Dockerfile	Base image	Build strategy
backend/Dockerfile	<code>node:20-alpine</code>	multi-stage : stage 1 installs <i>all</i> deps and compiles TypeScript → JS; stage 2 keeps only production deps + the compiled <code>dist/</code> . Smaller, fewer CVEs.
frontend/Dockerfile	<code>node:20-alpine</code> → <code>nginx:alpine</code>	stage 1 runs <code>expo export</code> to produce static web files and fails the build if no bundle came out; stage 2 is nginx serving that folder on port 8080 with the rules in frontend/nginx.conf — real 404s for missing hashed assets, SPA fallback only for routes.
backend-database/Dockerfile	<code>mongo:7-jammy</code>	official MongoDB + our <code>init-indexes.js</code> bootstrap script.
deploy/nginx/Dockerfile (compose only)	<code>nginx:alpine</code>	the reverse proxy, with <code>nginx.conf</code> copied into the image . Not bind-mounted: a single-file mount tracks the file's inode, so a <code>git pull</code> or an editor save silently leaves the container on the old config until it is recreated.

Key ideas used here

- **Multi-stage builds** — the compiler and dev dependencies never reach the final image.
- **Build context is the repo root for the backend** — because `backend/tsconfig.json` compiles `backend/` and `backend-database/` together. That is why the compose file and every build command say `docker build -f backend/Dockerfile .` (the `.` is the repo root).
- **Build-time vs run-time config**. `EXPO_PUBLIC_API_URL` is a **build arg** (`--build-arg`) because Expo inlines it into the JS bundle when it exports. Everything the *backend* needs (`MONGODB_URI`, `JWT_SECRET`, ...) is a **runtime env var**, injected when the container starts.
- **HEALTHCHECK** — every image knows how to answer "are you actually working?" (`GET /api/health` for the backend, `GET /` for the frontend, `db.adminCommand('ping')` for Mongo). Compose, Kubernetes, and ECS all use these to decide when a container is ready for traffic.
- **.dockerignore** — keeps `node_modules`, `.env`, and build output out of the build context so images are small and never bake in a secret.

3. NGINX — the reverse proxy

What "reverse proxy" means

A normal proxy sits in front of *clients*. A **reverse proxy** sits in front of *servers*: the outside world talks only to nginx, and nginx decides which internal service should handle each request. Benefits:

- **One public entry point** (one hostname, one port, one TLS certificate).
- **Path-based routing** — `/api` → backend, everything else → frontend.
- **The browser sees a single origin**, so no cross-origin (CORS) problems and cookies "just work".
- A natural place to add caching, rate limiting, gzip, and access logs later.

TrekEasy's routing (`deploy/nginx/nginx.conf`)

Request path	Sent to	Why it is separate
<code>/api/...</code>	<code>backend:3001</code>	the REST API
<code>/socket.io/...</code>	<code>backend:3001</code>	real-time chat; needs the WebSocket <code>Upgrade</code> headers and a long read timeout
<code>/uploads/...</code>	<code>backend:3001</code>	locally-stored images, served <i>outside</i> the <code>/api</code> prefix
<code>/</code> (everything else)	<code>frontend:8080</code>	the Expo web bundle

The exact same four rules appear in `k8s/ingress.yaml` and in the AWS ALB's listener rules. nginx, the Kubernetes Ingress, and the ALB are three implementations of one idea: *single front door, route by path*.

4. CI/CD — automated build, test, and deploy

CI (Continuous Integration) = every push is automatically built and tested, so breakage is caught in minutes, not in code review. **CD (Continuous Delivery/Deployment)** = a change that passes CI is automatically packaged and shipped to a running environment.

TrekEasy has **three** pipeline definitions doing the same job for different tools. You only need one; all three are kept working as reference.

4a. GitHub Actions — CI (`.github/workflows/ci.yml`)

Runs on **every push and every pull request**. Two parallel jobs (`backend` , `frontend`), each on Node 18 **and** 20:

```
checkout → npm ci → lint → typecheck → test → build → upload build artifact
```

If any step fails, the commit is marked **✖** and (for a PR) merging is blocked.

4b. GitHub Actions — CD (`.github/workflows/cd.yml`)

Runs on **push to main** (or manually, choosing `staging` / `production`):

```
test  ┌
      │ deploy:
      │   1. assume an AWS IAM role via OIDC (no long-lived keys)
      │   2. aws/scripts/build-and-push.sh → logs in to ECR, builds the 3
      │                                       images, pushes each tagged
      │                                       with the git SHA
      │   3. aws/scripts/deploy.sh          → reads the task definition
      │                                       currently on trekeasy-service,
      │                                       swaps in the new image tags,
      │                                       registers a new revision,
      │                                       updates the service, waits
      │                                       for it to stabilize
      └─ 4. post the result to Slack (#trekeasy), if configured
```

This ships a new version onto the cluster/service that already exists — it does not create or reconfigure any AWS resource. Deployment secrets/vars live in **GitHub** → **Settings** → **Secrets and variables** → **Actions** — never in the repo. See `RUNBOOK.md §4`.

4c. Jenkins ([Jenkinsfile](#))

The same stages expressed as a declarative Jenkins pipeline, for teams running a Jenkins server instead of (or alongside) GitHub Actions:

Stage	What it does
Checkout	pull the commit that triggered the build
Build	<code>npm ci</code> + compile backend and frontend
Test	<code>npm run typecheck</code> + <code>npm test</code> (backend), <code>typecheck</code> (frontend)
Archive	save <code>backend/dist/**</code> and <code>frontend/dist/**</code> as downloadable build artifacts
<i>post</i>	print success/failure, always <code>cleanWs()</code> to free disk

It expects a **Node 20 tool named `node20`** configured in *Jenkins* → *Manage Jenkins* → *Tools*. A production Jenkins setup would add `Push` and `Deploy` stages mirroring the GitHub CD job (build image → push to registry → roll the service).

The pipeline stages, in general terms

Stage	Question it answers	TrekEasy tooling
Build	Does it compile?	<code>npm ci && npm run build</code> , <code>docker build</code>
Test	Does it behave?	<code>jest</code> (88 backend unit tests), <code>tsc --noEmit</code> , <code>eslint</code>
Push	Where's the artifact?	<code>docker push</code> to ECR (<code>trekeasy-backend</code> etc.)
Deploy	How does it reach users?	register a new ECS task-definition revision and <code>update-service</code> → rolling update behind the ALB health check

5. Orchestration manifests — Kubernetes and ECS

Once you have images, something has to **run** them: keep the right number of copies alive, restart crashed ones, roll out new versions without downtime, and wire networking. That "something" is an **orchestrator**. TrekEasy ships configs for two.

5a. Kubernetes (`k8s/`) — for Minikube / any K8s cluster

File	Kind	Plain English
<code>namespace.yaml</code>	Namespace	a private folder (<code>myapp</code>) for all TrekEasy objects
<code>app-config.yaml</code>	ConfigMap	non-secret backend env vars (<code>NODE_ENV</code> , <code>CORS_ORIGINS</code> , ...)
<code>db-secret.yaml</code>	Secret	<code>MONGODB_URI</code> + JWT keys (base64, <i>not</i> encrypted — see the file's note)
<code>mongo-deployment.yaml</code> + <code>-service.yaml</code> + <code>mongo-pvc.yaml</code>	Deployment / Service / PersistentVolumeClaim	MongoDB (1 replica) with a 2 GiB disk that survives restarts
<code>backend-deployment.yaml</code> + <code>-service.yaml</code>	Deployment / Service	the API — 2 replicas , rolling updates, liveness/readiness probes on <code>/api/health</code>
<code>backend-hpa.yaml</code>	HorizontalPodAutoscaler	automatically scale the backend 2 → 6 pods when CPU > 70 %
<code>frontend-deployment.yaml</code> + <code>-service.yaml</code>	Deployment / Service	the Expo web bundle — 2 replicas, ClusterIP (only the Ingress can reach it)
<code>ingress.yaml</code>	Ingress	the single front door on <code>app.local</code> , same path routing as nginx

Core Kubernetes vocabulary: a **Pod** is one running container group; a **Deployment** keeps *N* identical Pods alive and does rolling updates; a **Service** is a stable internal name+IP load-balancing across a Deployment's Pods; an **Ingress** exposes Services to the outside world by hostname/path.

5b. AWS ECS on Fargate (`aws/`) — the target for this project

ECS (Elastic Container Service) is AWS's orchestrator. **Fargate** is the "serverless" mode: you never create or patch EC2 servers — you hand AWS a task definition and it finds capacity to run it. `launchType: FARGATE` throughout.

The cluster, service, ECR repositories, DocumentDB cluster, IAM roles, networking and Secrets Manager secrets **already exist** — they were created and are changed outside this repository (console / by hand), not by anything under `aws/` . This repo only ships new application versions onto them (see §4b above and [aws/README.md](#)); it does not provision, resize, or tear down any of it, and there is no CloudFormation here.

ECS vocabulary mapped to Kubernetes:

ECS	≈ Kubernetes	TrekEasy
Task definition	Pod spec	image, CPU/memory, ports, env, secrets, health check, log config for the containers running on <code>trekeasy-service</code>
Task	Pod	one running copy of a task definition
Service	Deployment	<code>trekeasy-service</code> — keeps the desired count of tasks alive, does rolling updates, registers them with the load balancer
Cluster	(the cluster)	<code>trekeasy-cluster</code> — just a namespace for the service
ALB + target groups	Ingress	single front door, path routing
Secrets Manager	Secret	JWT keys and the DocumentDB connection string, injected into the task definition as container secrets
Task execution role	—	lets the ECS agent pull images from ECR / read secrets / write logs
Task role	ServiceAccount (IRSA)	permissions for the app code itself

The database tier is **Amazon DocumentDB**, a managed service — not a container TrekEasy runs itself. The backend connects to it over TLS using Amazon's CA bundle (fetched at image build time in `backend/Dockerfile` ; see the `MONGODB_URI` `tlsCAFile` parameter), the same driver code path as the local/Compose/K8s Mongo connection.

`aws/scripts/deploy.sh` never needs to know the task definition's roles, secrets, subnets, or security groups: it reads whatever is *currently* registered on `trekeasy-service` , replaces only the image tags for the `trekeasy-backend` / `trekeasy-frontend` / `trekeasy-nginx` containers, and registers that as a new revision. Everything else about the task carries forward unchanged.

6. One change, five environments

You run...	It becomes...
<code>npm run start:dev</code>	local Node process, Mongo on <code>localhost</code>
<code>docker compose up</code>	4 containers on your laptop, nginx on <code>:80</code>
push to a branch	GitHub Actions CI: lint + typecheck + test + build
<code>kubectl apply -f k8s/</code>	2+2+1 pods on Minikube, Ingress on <code>app.local</code>
push to <code>main</code>	GitHub Actions CD → ECR → ECS Fargate rolling update behind the ALB

Same source, same Dockerfiles, same health checks, same routing rules — only the orchestrator changes.

Part 2 — DevOps Runbook

TrekEasy — DevOps Runbook ("What you should do")

Every exact command to set up, build, test, run, and deploy TrekEasy from scratch — locally, on Kubernetes, and on **AWS ECS Fargate** — plus the manual, click-in-the-console steps that **cannot** be scripted.

Commands are **Linux / macOS / WSL / Git-Bash** (`bash`). On native Windows PowerShell, swap `export X=y` for `$env:X="y"` and `$(cmd)` for `$(cmd)` (same) — or just use WSL.

Legend:  = manual action in a browser / console ·  = terminal command.

0. Install the tools (one time, per workstation)



```
# Node.js 20 (via nvm)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
nvm install 20 && nvm use 20
node -v          # v20.x
npm -v

# Docker Engine + Compose plugin (Docker Desktop on Mac/Windows)
docker --version
docker compose version

# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install
aws --version    # aws-cli/2.x

# jq (used by aws/scripts/deploy.sh)
sudo apt-get install -y jq      # or: brew install jq

# Only for the Kubernetes path:
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

1. Get the code and generate secrets



```
git clone <this-repo-url> TREKEASY-FINAL
cd TREKEASY-FINAL

# Two 64-hex-character JWT signing keys. Keep them somewhere safe.
openssl rand -hex 32      # -> JWT_SECRET
openssl rand -hex 32      # -> JWT_REFRESH_SECRET
```

2. Run it locally — no Docker (fastest inner loop)

You need a MongoDB somewhere. Quickest is a throwaway container:

```
❏
```

```
docker run -d --name trekeasy-database -p 27017:27017 mongo:7-jammy
```

Backend

```
❏
```

```
cd backend
cp .env.example .env
# edit .env:
#   MONGODB_URI=mongodb://localhost:27017/trekeasy
#   JWT_SECRET=<first key from step 1>
#   JWT_REFRESH_SECRET=<second key>
#   NODE_ENV=development
npm ci
npm run typecheck      # must pass
npm test               # 88 tests, must pass
npm run start:dev      # -> http://localhost:3001/api
```

Verify:

```
❏
```

```
curl -s http://localhost:3001/api/health      # {"status":"ok",...}
```

Frontend (second terminal)

```
❏
```

```
cd frontend
npm ci
npm run typecheck
npm run dev           # Expo dev server; press w for web, or scan the QR in Expo Go
```

Useful backend scripts

```
❏
```

```
cd backend
npm run seed:destinations      # load the trek catalogue into Mongo
npm run lint
```

3. Run the whole stack locally — Docker Compose



```
cd TREKEASY-FINAL
cp .env.example .env
# edit .env: set JWT_SECRET and JWT_REFRESH_SECRET (from step 1).
# Leave NODE_ENV=production as-is. Add the machine's LAN/VM address to
# CORS_ORIGINS if you will open the app by IP rather than on localhost.

docker compose up --build      # builds all four: -database / -backend / -frontend / -nginx
```

Open **http://localhost**.

`curl http://localhost/healthz` should print `trekeasy-nginx ok`. If it does not, the request is not reaching the proxy container at all — something else holds port 80 — and no change to `deploy/nginx/nginx.conf` will affect what you see.



Day-to-day:

```
docker compose ps          # status + health of the 4 containers
docker compose logs -f backend # tail one service
docker compose down        # stop
docker compose down -v      # stop AND wipe the Mongo volume
docker compose up -d --build backend # rebuild+restart just one service
```

Images produced (named consistently): `trekeasy-frontend:latest`, `trekeasy-backend:latest`, `trekeasy-database:latest`.

4. AWS setup — what already exists, and the one manual step left

The AWS side is **already provisioned and running**: `trekeasy-cluster`, `trekeasy-service`, the three ECR repos, the DocumentDB cluster, the task execution/task IAM roles, and the Secrets Manager secrets holding the JWT keys and the DocumentDB connection string. None of that is created or changed by anything in this repo — it was set up by hand in the Console and stays that way. `aws/scripts/` only ships new *application versions* onto it (\$5, \$6).

The one thing this repo's CD workflow still needs from a human is a way to authenticate to AWS from GitHub Actions.

4.1 Prerequisites for running the scripts yourself



Whoever runs `aws/scripts/*.sh` by hand needs the AWS CLI configured with credentials that can push to the three ECR repos and describe/update-service/register-task-definition/wait on `trekeasy-cluster` / `trekeasy-service`:

```
aws configure
aws sts get-caller-identity      # confirms you're authenticated
export AWS_REGION=ap-south-1
```

4.2 GitHub → AWS trust, for the CD workflow (skip if deploying only from your laptop)

The CD workflow authenticates to AWS with **OIDC** — no stored AWS keys.

1.  **IAM → Identity providers → Add provider** - Type: **OpenID Connect** - Provider URL: `https://token.actions.githubusercontent.com` - Audience: `sts.amazonaws.com`
2.  **IAM → Roles → Create role → Web identity** - Identity provider: the one just added - Audience: `sts.amazonaws.com` - Add a trust-policy condition so only *this* repo can assume it: `json "StringLike": { "token.actions.githubusercontent.com:sub": "repo:<ORG>/<REPO>:*" }` - Permissions: push/pull on the three ECR repos (**AmazonEC2ContainerRegistryPowerUser**, or a repo-scoped policy) plus `ecs:DescribeServices`, `ecs:DescribeTaskDefinition`, `ecs:RegisterTaskDefinition`, `ecs:UpdateService` and `iam:PassRole` on the task's execution/task roles, scoped to `trekeasy-cluster` / `trekeasy-service`. Name it `trekeasy-github-deploy`. Copy its ARN.
3.  **GitHub repo → Settings → Secrets and variables → Actions - New repository secret**: `AWS_DEPLOY_ROLE_ARN` = the role ARN - **New repository secret**: `SLACK_WEBHOOK_URL` = your Slack incoming webhook (optional) - **New repository variable**: `AWS_REGION` = `ap-south-1`

4.3 Changing a backend secret

JWT keys and the DocumentDB connection string live in **Secrets Manager** and are referenced by ARN from the task definition (`secrets` , not `environment`). Rotate a value in Secrets Manager, then force the service to pick it up:

```
aws secretsmanager put-secret-value --region ap-south-1 \
  --secret-id <secret-id> --secret-string '...'
aws ecs update-service --region ap-south-1 --cluster trekeasy-cluster \
  --service trekeasy-service --force-new-deployment
```

4.4 Environment-variable ownership (who sets what, where)

Variable	Where it lives in AWS	Set by
<code>JWT_SECRET</code> , <code>JWT_REFRESH_SECRET</code> , DocumentDB connection string	Secrets Manager, referenced from the task definition	set by hand in Secrets Manager (§4.3)
<code>CORS_ORIGINS</code> , <code>NODE_ENV</code> , <code>PORT</code> , <code>TRUST_PROXY</code>	backend container's <code>environment</code> in the task definition	set by hand on the task definition (console, or a new revision)
<code>EXPO_PUBLIC_API_URL</code>	baked into the frontend image at <code>build (/api)</code>	<code>build-and-push.sh --build-arg</code>
<code>CLOUDINARY_*</code>	optional — unset falls back to local disk on the backend	add to the task definition if you adopt Cloudinary
<code>AWS_DEPLOY_ROLE_ARN</code> , <code>SLACK_WEBHOOK_URL</code> , <code>AWS_REGION</code>	GitHub Actions secrets/variables	you, in the GitHub UI (§4.2)

5. Ship a new version to AWS ECS Fargate

The cluster, service, ECR repos, DocumentDB and IAM roles already exist (§4). This is the only AWS deployment flow this repo has — it's exactly what `.github/workflows/cd.yml` runs on every push to `main`.

Prereqs: §0 (tools), §4.1 done, shell has `AWS_REGION=ap-south-1` exported and `aws sts get-caller-identity` succeeds.

5.1 Build and push the three images to ECR



```
cd TREKEASY-FINAL
TAG=$(git rev-parse --short HEAD)

./aws/scripts/build-and-push.sh "$TAG"
# -> <acct>.dkr.ecr.ap-south-1.amazonaws.com/trekeasy-backend : <tag> + latest
# -> ... /trekeasy-frontend : <tag> + latest
# -> ... /trekeasy-nginx : <tag> + latest
```

5.2 Roll the service onto the new tag



```
./aws/scripts/deploy.sh "$TAG"
# reads the task def currently on trekeasy-service, swaps in the new image
# tags, registers a new revision, updates the service, and waits for
# `aws ecs wait services-stable`
```

5.3 Get the URL and smoke-test



Find the ALB's DNS name once (it doesn't change day to day):

```
aws elbv2 describe-load-balancers --region ap-south-1 \
  --query 'LoadBalancers[].DNSName' --output text
```

```
APP_URL=http://<alb-dns-name>
curl -s "$APP_URL/api/health" # {"status":"ok",...}
curl -sI "$APP_URL/" # 200, text/html (the Expo bundle)
```

5.4 Trek catalogue

Nothing to do — the backend seeds the destination catalogue on every boot (`DestinationsService.onModuleInit`, an idempotent upsert). Confirm once the backend is healthy:



```
curl -s "$APP_URL/api/destinations" | head -c 300
```

To force a refresh after editing `backend/src/data/trek-metadata.ts`, redeploy the backend (§5.1–5.2) — the new task re-runs the upsert.

6. Day-2 operations

Ship a new version (what CD does automatically on push to `main`)

See §5 — `build-and-push.sh "$TAG"` then `deploy.sh "$TAG"`.

Roll back



```
aws ecs describe-services --region ap-south-1 --cluster trekeasy-cluster \
  --services trekeasy-service --query 'services[0].taskDefinition'      # current revision

aws ecs update-service --region ap-south-1 --cluster trekeasy-cluster \
  --service trekeasy-service --task-definition <family>:<PREVIOUS_REVISION>

aws ecs wait services-stable --region ap-south-1 --cluster trekeasy-cluster \
  --services trekeasy-service
```

Logs

Find the log group(s) configured on the task definition, then tail them:

```
TASK_DEF=$(aws ecs describe-services --region ap-south-1 --cluster trekeasy-cluster \
  --services trekeasy-service --query 'services[0].taskDefinition' --output text)

aws ecs describe-task-definition --region ap-south-1 --task-definition "$TASK_DEF" \
  --query 'taskDefinition.containerDefinitions[].logConfiguration.options'

aws logs tail <log-group> --region ap-south-1 --follow
```

Scale



```
aws ecs update-service --region ap-south-1 --cluster trekeasy-cluster \
  --service trekeasy-service --desired-count 2
```

Change a backend env var / secret

- Non-secret (`CORS_ORIGINS`, `NODE_ENV`, ...): edit the container's `environment` on the task definition (console, or `describe-task-definition` → edit the JSON → `register-task-definition` → `update-service`, the same mechanic `aws/scripts/deploy.sh` uses for image tags).
- Secret (JWT keys, DocumentDB URI): see §4.3.

Open a shell in a running task (debugging)



```
aws ecs update-service --region ap-south-1 --cluster trekeasy-cluster \
  --service trekeasy-service --enable-execute-command --force-new-deployment
# then:
TASK=$(aws ecs list-tasks --region ap-south-1 --cluster trekeasy-cluster \
  --service-name trekeasy-service --query 'taskArns[0]' --output text)
aws ecs execute-command --region ap-south-1 --cluster trekeasy-cluster \
  --task "$TASK" --container trekeasy-backend --interactive --command "/bin/sh"
```

Scaling down / tearing down

This repo's scripts never delete or resize AWS infrastructure — scaling the service to 0, deleting the service/cluster, or decommissioning ECR repos/DocumentDB/secrets are account-owner actions taken by hand in the Console or CLI, outside this repo's scope.

7. Kubernetes (Minikube) — alternative to ECS



```
minikube start
eval $(minikube docker-env)      # build images straight into Minikube's Docker

docker build -f backend/Dockerfile      -t trekeasy-backend:latest .
docker build -f frontend/Dockerfile --build-arg EXPO_PUBLIC_API_URL=/api \
                                          -t trekeasy-frontend:latest frontend
docker build -f backend-database/Dockerfile -t trekeasy-database:latest backend-database

minikube addons enable ingress
minikube addons enable metrics-server

# db-secret.yaml ships with demo JWT keys - replace them first:
#   node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
kubectl apply -f k8s/

kubectl get pods -n myapp -w
echo "$(minikube ip) app.local" | sudo tee -a /etc/hosts
curl http://app.local/api/health
```

Rolling update / rollback: 

```
kubectl apply -f k8s/backend-deployment.yaml
kubectl rollout status deployment/backend -n myapp
kubectl rollout undo      deployment/backend -n myapp
```

8. Full "from zero" checklist

- [] \$0 tools installed (`node -v` , `docker --version` , `aws --version` , `jq --version`)
- [] \$1 repo cloned, two `openssl rand -hex 32` keys saved
- [] Local: backend `npm ci && npm test green; npm run start:dev; /api/health = ok`
- [] Local: frontend `npm ci && npm run typecheck; npm run dev loads`
- [] Compose: `.env` filled; `docker compose up --build`; `http://localhost` works
- [] 📄 \$4.1 `aws configure` , `aws sts get-caller-identity` , `AWS_REGION=ap-south-1` exported
- [] 📄 \$4.2 GitHub OIDC provider + `trekeasy-github-deploy` role + repo secrets/vars (if using CD)
- [] 📄 \$5.1 `./aws/scripts/build-and-push.sh "$TAG"`
- [] 📄 \$5.2 `./aws/scripts/deploy.sh "$TAG"` (registers a new revision, waits for `services-stable`)
- [] 📄 \$5.3 `curl $APP_URL/api/health = ok`
- [] 📄 \$5.4 trek catalogue present (`curl $APP_URL/api/destinations`)
- [] Push to `main` → CD builds, pushes, rolls `trekeasy-service` , posts to Slack (if configured)

9. Troubleshooting

Symptom	Likely cause	Fix
<code>deploy.sh</code> fails to find the service	wrong cluster/service name or region	confirm <code>ECS_CLUSTER=trekeasy-cluster</code> , <code>ECS_SERVICE=trekeasy-service</code> , <code>AWS_REGION=ap-south-1</code>
Service won't start, <code>CannotPullContainerError</code>	image tag not in ECR	run <code>build-and-push.sh</code> with that tag first
Backend task cycles, logs show <code>ECONNREFUSED / MongoServerSelectionError</code>	DocumentDB connection string or TLS CA bundle issue	check the backend's CloudWatch logs; confirm <code>global-bundle.pem</code> was fetched at build time (<code>backend/Dockerfile</code>) and the Secrets Manager connection string is current
Backend logs: <code>CORS_ORIGINS</code> is not set; <code>refusing all cross-origin</code>	<code>CORS_ORIGINS</code> empty on the task definition	set it on the backend container's <code>environment</code> (console, or a new task-def revision) to the ALB's public origin
ALB returns 503	no healthy targets	<code>aws elbv2 describe-target-health --target-group-arn <arn></code> ; check the container health check + security groups
Chat/socket disconnects every ~60 s	ALB idle timeout too low, or no stickiness for the <code>/socket.io/*</code> rule	check the ALB's idle timeout and the target group's stickiness/ <code>/socket.io/*</code> rule in the console
<code>expo export</code> OOMs in CI	Node heap	already handled by <code>node:20-alpine</code> ; if it recurs, raise the frontend build's runner size
CD fails at "Configure AWS credentials"	OIDC trust / role ARN	re-check \$4.2 trust policy <code>sub</code> matches <code>repo:<ORG>/<REPO>:*</code> and <code>AWS_DEPLOY_ROLE_ARN</code> secret
CD fails inside <code>deploy.sh</code> with a <code>jq</code> /IAM error	the deploy role is missing an ECS or <code>iam:PassRole</code> permission	see the permission list in \$4.2